
TravelFactory Technical Test - Internship

2 messages

YAIR Laura <Laura.Yair@travelfactory.fr>
To: "fransheska.er@gmail.com" <fransheska.er@gmail.com>

Sun, Jul 19, 2026 at 12:11 PM

Hi Fransheska,

Further to our conversation about your application for our new internship opportunity as a Full Stack Engineer, please see below more details about Travelfactory Group:

<http://www.travelfactory.fr/en/homepage>

<https://www.travelski.com/>

<https://www.locatour.com/>

Please find below the instructions for the assignment.

You may complete the home assignment using either your own Bitbucket or GitHub account.

Please include a README file describing the project and explaining how to run it.

Please send me an email when you start working on the assignment, and another email once you have completed it.

Web Development Intern Recruitment Test: Vacation Management Interface

Objective

Develop a web application to manage vacation requests.

The application must include two user interfaces:

1. **Requester Interface**: Allows employees to submit vacation requests.
2. **Validator Interface**: Allows managers to review, approve, or reject requests.

The project must use:

- **Vue.js** for the frontend
- **Node.js** for the backend
- A **PostgreSQL** relational database with **TypeORM**
- A modern CSS framework such as **Tailwind CSS**
- A **Common event framework** <https://www.npmjs.com/package/commoneventframework?activeTab=readme>

Deliverables

1. A working application running locally, with clear setup instructions.

2. Well-structured and documented code, along with a brief explanation of your approach.
3. A public or private Github /Bitbucket repository.

Requirements

Authentication

Implement a simple authentication system.

Requirements:

- Login endpoint
- Protected routes
- Role-based access control
- Two roles:
 - Requester
 - Validator

You may use JWT authentication.

Seeded demo users are acceptable.

Requester Interface

Features

1. A form to submit vacation requests with the following fields:

- Start Date (required)
- End Date (required)
- Reason (optional)

2. A list displaying the user's submitted requests.

3. Request statuses:

- Pending
- Approved
- Rejected

4. Team vacation planning available for two roles.

Validator Interface

Features

1. A dashboard displaying all vacation requests.

2. Ability to filter requests by:

- Status
- User

3. Pagination for request listing.

4. Actions to:

- Approve a request
- Reject a request

5. A comment field is required when rejecting a request.

Business Rules

Implement the following validation rules:

1. End date must be after start date.
2. Vacation requests cannot overlap for the same user.
3. Approved requests cannot be modified.
4. Requests in the past are not allowed.
5. A rejected request must contain a rejection comment.
6. Only validators can approve or reject requests.

Backend API (Node.js)

Provide RESTful endpoints for:

1. Authentication
2. Submitting a vacation request
3. Retrieving vacation requests:
 - By user (requester view)
 - All requests (validator view)
4. Approving or rejecting a request

Additional requirements:

- Input validation
- Proper error handling
- RESTful best practices
- Pagination support
- Role-based authorization

Architecture Expectations

The project must demonstrate:

- Clean project structure
- Separation of concerns

- Command/Event-oriented architecture
- Modular backend design
- Reusable frontend components

Examples:

- Commands for actions such as CreateVacationRequestCommand or ApproveVacationRequestCommand
- Events such as VacationRequestApprovedEvent

You may use any implementation approach as long as the architecture is clearly visible.

Database

Users

- id
- name
- email
- password
- role (Requester / Validator)
- created_at

VacationRequests

- id
- user_id
- start_date
- end_date
- reason
- status (Pending / Approved / Rejected)
- comments
- created_at
- updated_at

Frontend Requirements

- Use Vue Router
- Use Axios for API calls
- Use Tailwind CSS or another modern CSS framework
- Responsive behavior is appreciated but advanced design is not required
- Focus on usability, clarity, and code quality rather than visual creativity

Evaluation Criteria

1. Code Quality

- Clean, modular, and maintainable code
- Proper naming and structure
- Good separation of concerns

- Consistent formatting

2. Architecture

- Proper use of commands/events
- Good API organization
- Scalable backend structure

3. Functionality

- Requirements fully implemented
- Correct handling of edge cases
- Validation rules enforced

4. Database Design

- Clean schema
- Efficient relations and queries
- Proper pagination support

5. Documentation

- Clear README
- Setup instructions
- Technical decisions
- Known limitations

Instructions for the Candidate

Include a README.md containing:

- Installation instructions
- Run instructions
- Database setup
- Technical decisions
- Known limitations

Use Git with meaningful commits.

Estimated Time with AI

Expected effort: **2 to 3 hours**.

Good luck 😊

Laura YAIR

Executive & Hr Assistant - Travelfactory

laura.yair@travelfactory.fr

Wopa Tower - [HaSheizaf Street 4](#) - 4366411 Ra'anana - Israel



 Adoptez l'éco-attitude ! N'imprimez cet email que si c'est nécessaire.

Laura YAIR

Executive & Hr Assistant - Travelfactory

laura.yair@travelfactory.fr

Wopa Tower - [HaSheizaf Street 4](#) - 4366411 Ra'anana - Israel



 Adoptez l'éco-attitude ! N'imprimez cet email que si c'est nécessaire.

Fransheska Echevarria <fransheska.er@gmail.com>

To: YAIR Laura <Laura.Yair@travelfactory.fr>

Sun, Jul 26, 2026 at 3:12 PM

Hi Laura,

Thanks for sending over the assignment. As I mentioned on the phone, I had a final exam last week, so I'm starting to work on it today. I'll email you again as soon as it's completed.

Best,

Fransheska Echevarria

www.linkedin.com/in/fransheskaer

[Quoted text hidden]